

מבוא לתכנות מערכות – מבחן מועד ב', סתיו תשע"ז

מרצה: ד"ר איתי שרון

משך המבחן: 180 דקות

חומר עזר מותר: 2 דפים המכילים תאור פקודות ב-C ו-bash. כל חומר עזר אחר אסור.

המבחן כולל 5 שאלות, יש לענות על כולן. בשאלות הדורשות נימוק הקפידו לספק אותו.

יש לענות על טופס הבחינה במקומות המיועדים לכך. ניתן להשתמש במחברת העזר אולם יש להגיש רק את טופס הבחינה.

שני העמודים האחרונים במבחן כוללים חומר שיכול (אך לא חייב!) לעזור לכם.

בהצלחה!

```

1 void lolfunction(char *s) {
2     int n = 0;
3     while(*(s+n))
4         n++;
5     n--;
6     while(n>0) {
7         *s ^= *(s+n);
8         *(s+n) ^= *s;
9         *s ^= *(s+n);
10        s++; n-=2;
11    }
12 }
```

הפונקציה הופכת את המחרוזת מהסוף להתחלה. למשל hello יהפוך ל-olleh.

ב. ממשו ב-C גרסא מצומצמת של הפקודה cut אשר מקבלת מצביע לקובץ (fp), אינדקס התחלה (s) ואינדקס סוף (e), ומדפיסה עבור כל שורה בקובץ המוצבע ע"י fp את כל התווים בין s ל-e כולל התווים באינדקסים אלה. ניתן להניח ש- $e \geq s \geq 0$ כאשר 0 הוא האינדקס של התו הראשון. עבור שורות באורך n שעבורן $s \geq n$ תדפיס הפונקציה שורה ריקה (רק '\n'). אחרת אם $e \geq n \geq s$ הפונקציה תדפיס את כל התווים בין s ל-1-n. אחרת תדפיס הפונקציה את כל התווים בין s ל-e (כולל).

```
void my_cut(FILE* fp, int s, int e) {
```

```
    char* buf = NULL;
```

```
    size_t bufsize = 0;
```

```
    int i, n;
```

```
    while((n = getline(&buf, &bufsize, fp)) > 0) {
```

```
        for(i=s; i<=e && i<n; i++)
```

```
            putchar(buf[i], stdout);
```

```
        if(i < n) // If we've reached n then we also printed \n
```

```
            putchar('\n', stdout);
```

```
    }
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 2 (30 נקודות)

בשאלה זו נבנה מבנה נתונים בשם Flights שמטרתו היא שמירת נתונים עבור מחירי טיסות בין נמלי תעופה ותכנון מסלול הטיסה הזול ביותר. מבנה הנתונים יכיל עבור כל טיסה את הנתונים הבאים:

- נמל התעופה ממנו יוצאת הטיסה. לכל נמל תעופה קוד משלו אשר נתון על ידי מזהה מטיפוס int.
- נמל התעופה אליו מגיעה הטיסה (קוד מטיפוס int).
- מחיר הטיסה (מספר מטיפוס int)
- שם חברת התעופה (מחרוזת)

הקודים של נמלי התעופה נעים בין 0 ל-1-NUM_AIRPORTS כאשר NUM_AIRPORTS הוא מספר שדות התעופה הקיימים בעולם (כ-44 אלף על פי גוגל, הניחו שקבוע זה מוגדר). הניחו גם שלכל חברה יש לכל היותר סוג אחד של טיסות בין שני שדות תעופה.

א. כתבו ממשק ADT עבור Flights אשר מאפשר את הפעולות הבאות:

- **יצירת אובייקט Flights חדש**
- **הוספת טיסה.** הנתונים שיתקבלו הם שדה התעופה ממנו יוצאים, שדה התעופה אליו מגיעים, שם חברת התעופה והמחיר.
- **הסרת טיסה.** הנתונים שיתקבלו הם שדה התעופה ממנו יוצאים, שדה התעופה אליו מגיעים ושם חברת התעופה.
- **קבלת רשימת שדות התעופה אליהם יש טיסה ישירה משדה תעופה מסויים.** המידע יועבר על ידי מערך של int.
- **קבלת המחיר הזול ביותר לטיסה ישירה בין שני נמלי תעופה.** אם לא קיימת טיסה כזאת תחזיר הפונקציה את הקבוע NO_FLIGHTS שיוגדר להיות המספר הגדול ביותר אותו ניתן לאכסן על ידי int (הניחו שקבוע זה מוגדר).
- **קבלת המחיר הזול ביותר למסלול עם בדיוק n טיסות בין שני נמלי תעופה.** אם אין מסלול כזה יוחזר NO_FLIGHTS.

```
#ifndef FLIGHTS_H
```

```
#define FLIGHTS H
```

```
typedef struct Flights* Flights;
```

```
Flights FlightsCreate();
```

```
void FlightAdd(Flights f, int from, int to, const char* company, int price);
```

```
void FlightRemove(Flights f, int from, int to, const char* company);
```

```
void AccessibleAirports(Flights f, int from, int** airports, int* size);
```

```
int CheapestFlight(Flights f, int from, int to);
```

```
int CheapestNFlights(Flights f, int from, int to, int nflights);
```

```
#endif
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. בחרו במבנה הנתונים המתאים ביותר מתוך אלו שראינו בכתה וכתבו את המבנים הדרושים למימוש Flights. ממשו את הפונקציה אשר מחזירה את רשימת נמלי התעופה אליהם ניתן להגיע על ידי טיסה ישירה מנמל תעופה מסויים.

```
typedef struct Flight* Flight;

struct Flight {
    int from, to, fare;
    const char* company;
    Flight next;
};

struct Flights {
    Flight all flights; // Implemented as a linked list
};

// This implementation may return the same airport several times. I was
// fine with that while checking the test

void AccessibleAirports(Flights f, int from, int** airports, int* size) {
    *size = 0;
    Flight flight;
    for(flight=f->all flights; flight!=NULL; flight=flight->next)
        (*size) += (flight->from == from);
    (*airports) = (int*)malloc(sizeof(int)*(*size));
    // Make sure allocation succeeded etc - not shown
    *size = 0;
    for(flight=f->all flights; flight!=NULL; flight=flight->next)
        if(flight->from == from)
            (*airports)[(*size)++] = flight->to;
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ג. ממשו את הפונקציה אשר מחזירה את המחיר הזול ביותר עבור מסלול עם בדיוק n עצירות.

```
int CheapestNFlights(Flights f, int from, int to, int nflights)
{
    Flight l;

    int lowest fare = NO FLIGHTS;

    if(num flights <= 0)
        return NO FLIGHTS;

    for(l=f->all flights; l!=NULL; l=l->next) {
        if((l->to == to) && (num flights == 1) && (l->fare < lowest fare)) {
            lowest fare = l->fare;
        }
        else {
            int rest = CheapestNFlights(f, l->to, to, num flights-1);
            if((rest != NO FLIGHTS) && (l->fare+rest < lowest fare))
                lowest fare = l->fare+rest;
        }
    }

    return lowest fare;
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 3 (10 נקודות)

כתבו פונקציה אשר מוצאת את מספר האחדים והאפסים בייצוג הבינארי של מספר אך לא עושה שימוש בפעולות של ביטים. ניקוד מופחת יינתן על מימוש שכולל פעולות ביטים. למשל: עבור המספר 15 תחזיר הפונקציה 4 (15=1111b)

```
int numOnes(unsigned int n) {  
  
    int nones=0;  
  
    while(n>0) {  
  
        nones += n%2;  
  
        n /= 2;  
  
    }  
  
    return nones;  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 4 (20 נקודות)

שאלה זו עוסקת בעצי חיפוש בינארים.

נתון ממשק חלקי של עץ חיפוש בינארי בדומה למה שראינו בכתה עבור מספרים שלמים:

```
typedef struct BNode* BNode;
struct BNode {
    int data;
    BNode left, right;
};

BNode BSTAddNode(BNode, int);
BNode BSTRemoveNode(BNode, int);
bool BSTSearchNode(BNode, int);
void BSTpreorder(BNode);
void BSTinorder(BNode);
void BSTpostorder(BNode);
```

א. ציירו את העץ המתקבל מהכנסת המספרים הבאים בסדר בו הם מופיעים (משמאל לימין). כמו כן כתבו את תוצאת הרצת הפונקציה preorder על העץ המתקבל.

11, 33, 2, 8, 68, 243, 18, 40

11

2

33

8

18

68

40

243

Preorder: 11, 2, 8, 33, 18, 68, 40, 243

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. ממשו את הפונקציה nth אשר מקבלת מצביע לעץ חיפוש בינארי ומדפיסה למסך את האיבר ה-n בגודלו. למשל: עבור העץ מסעיף א, אם נקרא לפונקציה עם $n=3$, הפונקציה תדפיס את המספר 11. אם n המתקבל הוא קטן מ-1 או גדול ממספר האיברים בעץ לא תדפיס הפונקציה דבר. נתונה חתימת הפונקציה. הפונקציה מחזירה int (במימוש שלי לפונקציה זה עזר), אתם רשאים לשנות טיפוס זה אם אתם רוצים.

```
int nth(BNode node, int n) {  
  
    int nones=0;  
  
    if(node == NULL)  
  
        return n;  
  
    n = nth(node->left, n);  
  
    if(n < 1)  
  
        return n;  
  
    if(n == 1) {  
  
        printf("%d\n");  
  
        return 0;  
  
    }  
  
    return nth(node->right, n-1);  
}
```


המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 5 (20 נקודות)

א. נתון קובץ טקסט פשוט בשם poem.txt אשר מכיל את השיר הבא (המספרים מצד שמאל אינם חלק מהטקסט אלא באים רק לסמן את מספר כל שורה). כל הרווחים הם רווחים רגילים (ולא טאבים), שורות שנראות ריקות הן אכן ריקות.

1	The lobster and the crab one day
2	Proposed a friendly race.
3	Agreed upon the time were they,
4	Agreed upon the place.
5	
6	The start and finish lines were where
7	The two thought they should be.
8	The crayfish with a clock was there
9	To act as referee.
10	
11	And though the rule-book then was read,
12	Not all was clarified;
13	For as the lobster forward sped
14	The crab went to the side.
15	
16	By Jeffrey Krise

כתבו את מספרי השורות אשר יודפסו כתוצאה מהפקודות הבאות.

egrep ace.\$ poem.txt	2, 4
egrep ^\$ poem.txt	5, 10, 15
egrep "[A-Z]" poem.txt	16
egrep ^T.*e\$ poem.txt	6, 8
egrep e{2} poem.txt	3, 4, 9

מה תדפסנה הפקודות הבאות?

```
egrep crab poem.txt | grep lobster | sed 's/lobster/crab/; s/crab/lobster/'
```

The lobster and the crab one day

```
egrep t....$ poem.txt | cut -f2,6,7 -d" "
```

upon they,

crayfish was there

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו סקריפט אשר מקבל שם של תיקייה, עובר על התוכן שלה ומדפיס עבור כל פריט:

- אם הפריט הוא קובץ – הסקריפט ידפיס שורה הכוללת את שם הקובץ, לאחריו המילה file, ולאחריו מספר התווים בקובץ
- אם הפריט הוא תיקייה – הסקריפט ידפיס שורה הכוללת את שם התיקייה, המילה dir ומספר הפריטים בתיקייה.
- בסיום תודפס שורת סיכום המתארת כמה קבצים ותיקיות היו בתיקייה שהועברה.

למשל: עבור התיקייה tmp אשר מכילה שלושה קבצים (foo.txt, bar.txt ו-poem.txt) ותיקייה ריקה אחת ידפיס הסקריפט:

```
> ./5b.sh tmp
tmp/bar.txt      file 1 bytes
tmp/empty_dir   dir 0 items
tmp/foo.txt      file 1 bytes
tmp/poem.txt     file 386 bytes
```

```
3 files, 1 directories in tmp
```

בידקו שהועבר פרמטר אחד לסקריפט אך במידה והוא הועבר אפשר להניח שהוא מתאר תיקייה חוקית.

```
#!/bin/bash
```

```
if [[ ! $# -eq 1 ]]; then
```

```
    echo -e "\nUsage: $0 <root-directory>\n"
```

```
    exit
```

```
fi
```

```
nfiles=0
```

```
ndirs=0
```

```
for x in $1/*; do
```

```
    if [[ -f $x ]]; then
```

```
        b=`cat $x | wc -c | sed -E 's/ +//'`
```

```
        echo -e "$x\tfile\t$b bytes"
```

```
        ((nfiles++))
```

```
    else
```

```
        n=`ls $x | wc -l | sed -E 's/ +//'`
```

```
        echo -e "$x\tdir\t$n items"
```

```
        ((ndirs++))
```

```
    fi
```

```
done
```

```
echo -e "\n$nfiles files, $ndirs directories in $1"
```

[illegible]

המכללה האקדמית תל חי, החוג למדעי המחשב

פקודות bash

- **cat** – מדפיסה תוכן של קובץ למסך
- **cut** – עבור כל שורה בקלט מדפיסה שדות ספציפיים (-f) המופרדים ע"י טאב או ע"י מפריד אחר הנתון ע"י הפרמטר -d, או עמודות ספציפיות (-b)
- **egrep** – זהה ל-grep -e
- **find** – מחזירה רשימה של כל הקבצים אשר מתאימים לתאור המשתמש (ארגומנט -name) בספרייה מסוימת ואלו שנמצאות תחתיה (ארגומנט ראשון לפקודה)
- **grep** – מחזירה שורות אשר מכילות string מסויים. ניתן להשתמש ב-regular expressions (-e), ולהחזיר שורות שאינן מכילות את ה-string אותו מחפשים (-v)
- **head** – מדפיסה את 10 השורות הראשונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב-n)
- **ls** – מדפיסה תוכן ספרייה, או רשימה של קבצים אשר מתאימה לתאור שהעביר המשתמש כארגומנט (למשל: ls *.c תדפיס את שמות הקבצים שמסתיימים ב-c. בספרייה הנוכחית)
- **read** – קוראת שורה מה-standard input ומכניסה אותה למשתנה שמועבר ע"י המשתמש כארגומנט
- **sed** – עורכת טקסט הנתון כקלט. הפעולה s תשנה שורת טקסט.
- **sort** – ממיינת לקסיקוגרפית קובץ קלט. ניתן למיין מספרית (-n) ולשמור רק עותק אחד של שורות שחוזרות על עצמן (-u)
- **tail** – מדפיסה את 10 השורות האחרונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב-n)
- **wc** – מחזירה את מספר השורות (-l), בתים (-c) או מילים (-w) בקובץ

פעולות על ביטים

- bitwise AND - &
- bitwise OR - |
- bitwise XOR - ^
- right shift - >>
- left shift - <<
- One's complement - ~

פונקציות ב-C

- **int putchar(int character, FILE* fp)** – כותבת את character ל-fp.
- **int fgetc(FILE* fp)** – קוראת תו אחד מתוך fp.
- **scanf(const char* format,...)** – קוראת מידע מתוך stdin בהתאם ל-format, מאחסנת את המידע בכתובות המשתנים שמסופקים. **fscanf(FILE* stream, const char* format,...)** – אותו דבר כאשר הקלט נקרא מתוך stream.
- **sscanf(const char* s, const char* format,...)** – אותו הדבר כאשר הקלט נקרא מתוך s.
- **int getline(char** lineptr, size_t* n, FILE* stream)** – קוראת שורה לתוך lineptr, מקצה זכרון עבור lineptr אם אין מספיק (גודל ה-buffer הנוכחי מצוי ב-n). מחזירה את מס' התווים שקראה או -1 אם לא הצליחה לקרוא.
- **printf(const char* format,...)** – כתיבת מידע ערוך ל-stdout בהתאם ל-format והמשתנים שמסופקים.
- **fprintf(FILE* stream, const char* format,...)** – אותו דבר כאשר הכתיבה נעשית לתוך stream.
- **sprintf(const char* s, const char* format,...)** – אותו הדבר כאשר הכתיבה נעשית לתוך s.
- **void* malloc(size_t size)** – מקצה זכרון בגודל size
- **void* calloc(size_t num, size_t size)** – מקצה num יחידות בגודל size כל אחת ומאפסת את כל הזכרון שהוקצה.
- **void* realloc(void* ptr, size_t size)** – מקצה size בתים, מוסיפה/מורידה מכמות הזכרון ש-ptr מצביעה עליו (צריך להיות מוקצה דינמית גם כן), עשויה לשנות את הזכרון המוקצה ואז תעתיק את התוכן של ptr לשטח החדש. מחזירה את כתובת הזכרון שהוקצה.

המכללה האקדמית תל חי, החוג למדעי המחשב

- `const char* strstr(const char* str1, const char* str2)` – מחזירה את המקום שבו מופיעה `str2` ב-`str1` (או NULL אם היא לא מופיעה)
 - `int strcmp(const char* str1, const char* str2)` – משווה את `str1` ל-`str2`, מחזירה 0 אם הן זהות, ערך קטן מ-0 אם `str1` קטן לקסיקוגרפית מ-`str2` או ערך גדול מ-0 אחרת.
 - `size_t strlen(const char* str)` – מחזירה אורך של מחרוזת.
 - `int fseek(FILE *fptr, long offset, int origin)` – מזיזה את `fptr` ל-`offset` בתיים ביחס ל-`origin`. המשתנה `origin` יכול להיות
 - `SEEK_SET` – תחילת הקובץ
 - `SEEK_CUR` – מיקום נוכחי
 - `SEEK_END` – סוף הקובץ
- הפונקציה תחזיר 0 בהצלחה או ערך שונה מ-0 אחרת.